Hash Tables

Comp Sci 214, Spring 2025

Don’t forget to check in on Youhere!

Dictionary data structures, with lookup times

- Sorted array
  - Binary search — $\mathcal{O}(\log n)$
- Association list
  - Linear search — $\mathcal{O}(n)$

Dictionary data structures, with lookup times

- Sorted array
  - Binary search — $\mathcal{O}(\log n)$
- Association list
  - Linear search — $\mathcal{O}(n)$

Now what if we had a special case? Keys $\in \{0, 1, \ldots, k-1\}$

We can do better!

Dictionary data structures, with lookup times

- Sorted array
  - Binary search — $\mathcal{O}(\log n)$
- Association list
  - Linear search — $\mathcal{O}(n)$

Now what if we had a special case? Keys $\in \{0, 1, \ldots, k-1\}$

We can do better!

- An array of size k, using keys as indices
  - Array indexing — $\mathcal{O}(1)$!

This technique is known as *direct addressing*

Direct addressing example

Suppose we want to map digits to their names in English:

```
let digits = ['zero', 'one', 'two', 'three', 'four',
              'five', 'six', 'seven', 'eight', 'nine']

def get_digit_name(i: nat?) -> str?:
    return digits[i]
```

If these are the keys you need, **excellent** data structure.

Non-direct addressing example

A phone book is a dictionary where the keys are names of people and the values are their phone numbers.

Need to somehow find a way to use strings as keys for an array.

Non-direct addressing example

A phone book is a dictionary where the keys are names of people and the values are their phone numbers.

Need to somehow find a way to use strings as keys for an array.

Let's map these strings to integers $\in \{0, 1, \ldots, k-1\}$!

We call a function that maps keys to integers a *hash function*.

We call its output a *hash code* (or just a *hash*).

We call the array we use afterwards a *hash table*, and we call its elements *buckets*.

Hash functions are *one-way*: can't derive inputs from outputs.

Non-direct addressing example

A phone book is a dictionary where the keys are names of people and the values are their phone numbers.

Need to somehow find a way to use strings as keys for an array.

Let's map these strings to integers $\in \{0, 1, \ldots, k-1\}$!

We call a function that maps keys to integers a *hash function*.

We call its output a *hash code* (or just a *hash*).

We call the array we use afterwards a *hash table*, and we call its elements *buckets*.

Hash functions are *one-way*: can't derive inputs from outputs.

First Attempt: use the position in the alphabet of the first letter of the person's name as our integers (A=0, B=1, …)

The “first letter” hash table

We have an array of “buckets”, indexed by the hash of the name, each of which can hold a single key-value association.

(bucket)	name	phone
(0)	`"Allison"`	555–1212
(1)	⊘	⊘
(2)	`"Carol"`	555–1214
(3)	⊘	⊘
⋮		
(24)	`"Youssef"`	555–1215
(25)	⊘	⊘

To look up a key: hash the key, look in the correct bucket. If it has the right key, we found the value.

To insert a key: hash the key, add key and value to the correct bucket.

The “first letter” hash table

We have an array of “buckets”, indexed by the hash of the name, each of which can hold a single key-value association.

(bucket)	name	phone
(0)	`"Allison"`	555–1212
(1)	⊘	⊘
(2)	`"Carol"`	555–1214
(3)	⊘	⊘
⋮		
(24)	`"Youssef"`	555–1215
(25)	⊘	⊘

Q: If we insert Charles, which bucket would he map to?

To look up a key: hash the key, look in the correct bucket. If it has the right key, we found the value.

To insert a key: hash the key, add key and value to the correct bucket.

Hash collision!

Using our “first letter” hash function h_1:

$$h_1(\texttt{"Allison"}) = 0$$
$$h_1(\texttt{"Carol"}) = 2$$
$$h_1(\texttt{"Youssef"}) = 24$$
$$h_1(\texttt{"Charles"}) = \textit{also } 2!$$

When the hash function gives the same value for two keys, that’s called a *hash collision*.

And we need some way to *resolve* the collision.

Hash collision!

Using our "first letter" hash function h_1:

$$h_1(\texttt{"Allison"}) = 0$$
$$h_1(\texttt{"Carol"}) = 2$$
$$h_1(\texttt{"Youssef"}) = 24$$
$$h_1(\texttt{"Charles"}) = \textit{also } 2!$$

When the hash function gives the same value for two keys, that's called a *hash collision*.

And we need some way to *resolve* the collision.

Note: while h_1 is a terrible hash function (stay tuned), collisions are inevitable regardless of hash function.

Solutions to handle hash collisions

1. Store a linked list in each bucket (**separate chaining**)
   - That's part of homework 3

Solutions to handle hash collisions

1. Store a linked list in each bucket (**separate chaining**)
   - That's part of homework 3

2. Use some other free bucket instead (**open addressing**)
   - Use the next free bucket (**linear probing**) – we'll see today
   - Try further and further buckets (**quadratic probing**)
   - Do more hashing with a different hash function to find another bucket (**double hashing**) – see Chapter 8

Solutions to handle hash collisions

1. Store a linked list in each bucket (**separate chaining**)
   - That's part of homework 3

2. Use some other free bucket instead (**open addressing**)
   - Use the next free bucket (**linear probing**) – we'll see today
   - Try further and further buckets (**quadratic probing**)
   - Do more hashing with a different hash function to find another bucket (**double hashing**) – see Chapter 8

Plus variants on top of these options:

- Robin Hood hashing
- 2-choice hashing
- Fibonacci hashing
- etc. (very deep topic!)

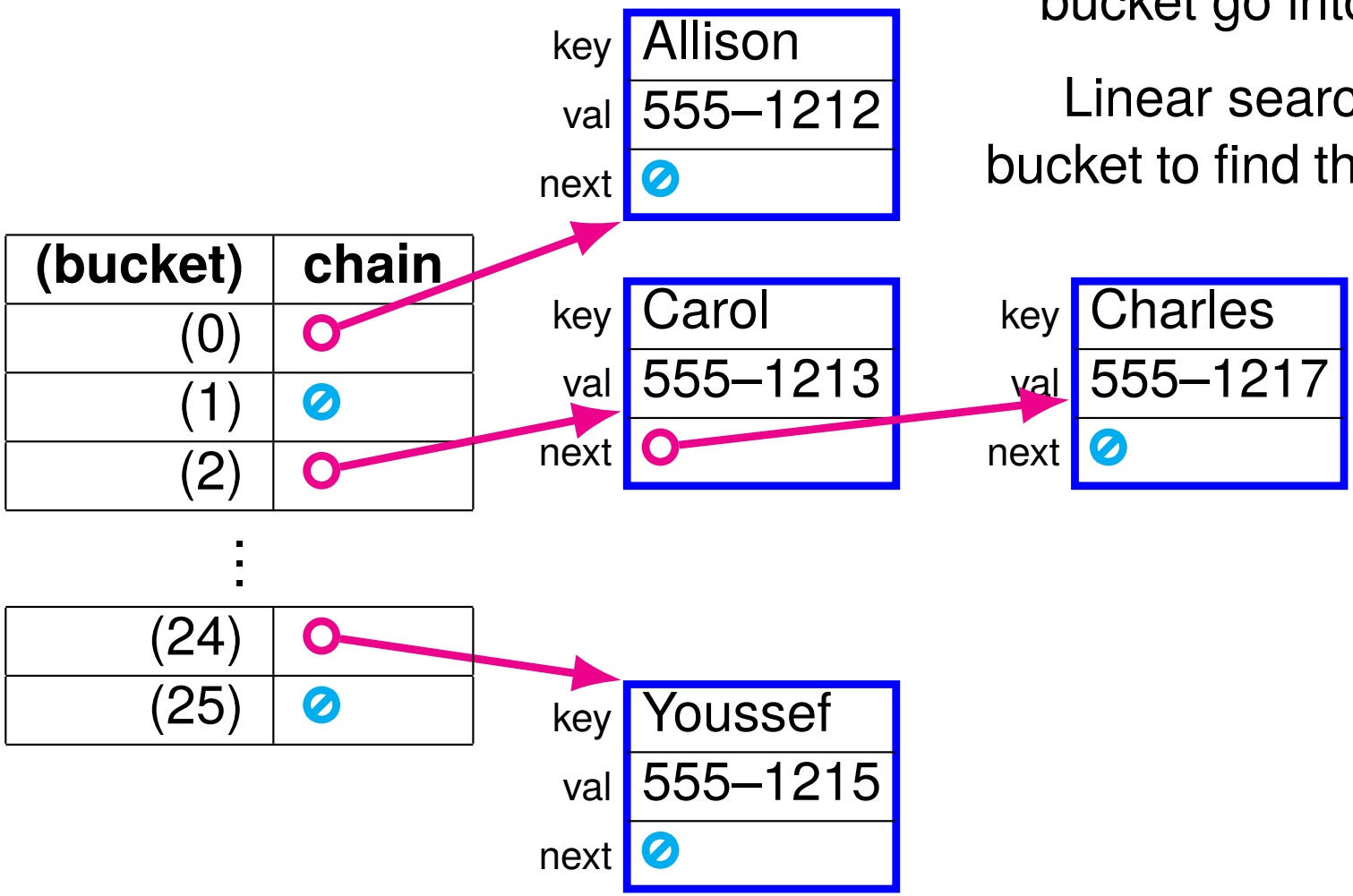
Lots of different approaches, all with different tradeoffs!

- But all are the same data structure (hash table)!

Separate chaining hash table

Each bucket is a list! All the keys that map to that bucket go into the list.

Linear search in the bucket to find the right key.



Linear probing hash table

If your bucket is taken, try the next one! (then the next, …)
→ Number of elements limited by the number of buckets.

(bucket)	name	phone
(0)	`"Allison"`	555–1212
(1)	⊘	⊘
(2)	`"Carol"`	555–1214
(3)	⊘	⊘
(4)	⊘	⊘
⋮		
(24)	`"Youssef"`	555–1215
(25)	⊘	⊘

Lookup: start at “correct” bucket, keep looking until we find the right key (or find an empty spot → key must be absent)

Linear probing hash table

If your bucket is taken, try the next one! (then the next, …)
→ Number of elements limited by the number of buckets.

(bucket)	name	phone
(0)	`"Allison"`	555–1212
(1)	⊘	⊘
(2)	`"Carol"`	555–1214
(3)	⊘	⊘
(4)	⊘	⊘
⋮		
(24)	`"Youssef"`	555–1215
(25)	⊘	⊘

Let's insert Charles.

Lookup: start at "correct" bucket, keep looking until we find the right key (or find an empty spot → key must be absent)

Linear probing hash table

If your bucket is taken, try the next one! (then the next, …)
→ Number of elements limited by the number of buckets.

(bucket)	name	phone
(0)	`"Allison"`	555–1212
(1)	⊘	⊘
(2)	`"Carol"`	555–1214
(3)	⊘	⊘
(4)	⊘	⊘
⋮		
(24)	`"Youssef"`	555–1215
(25)	⊘	⊘

Let's insert Charles.
Bucket 2 is taken.

Lookup: start at "correct" bucket, keep looking until we find the right key (or find an empty spot → key must be absent)

Linear probing hash table

If your bucket is taken, try the next one! (then the next, …)
→ Number of elements limited by the number of buckets.

(bucket)	name	phone
(0)	`"Allison"`	555–1212
(1)	⊘	⊘
(2)	`"Carol"`	555–1214
(3)	`"Charles"`	555–1217
(4)	⊘	⊘
⋮		
(24)	`"Youssef"`	555–1215
(25)	⊘	⊘

Let's insert Charles.
Bucket 2 is taken.
Next bucket is 3, it's free.
Let's put Charles there!

Lookup: start at "correct" bucket, keep looking until we find the right key (or find an empty spot → key must be absent)

Linear probing hash table

If your bucket is taken, try the next one! (then the next, …)
→ Number of elements limited by the number of buckets.

(bucket)	name	phone
(0)	`"Allison"`	555–1212
(1)	⊘	⊘
(2)	`"Carol"`	555–1214
(3)	`"Charles"`	555–1217
(4)	⊘	⊘
⋮		
(24)	`"Youssef"`	555–1215
(25)	⊘	⊘

Let's look up Charles.

Lookup: start at "correct" bucket, keep looking until we find the right key (or find an empty spot → key must be absent)

Linear probing hash table

If your bucket is taken, try the next one! (then the next, …)
→ Number of elements limited by the number of buckets.

(bucket)	name	phone
(0)	`"Allison"`	555–1212
(1)	⊘	⊘
(2)	`"Carol"`	555–1214
(3)	`"Charles"`	555–1217
(4)	⊘	⊘
⋮		
(24)	`"Youssef"`	555–1215
(25)	⊘	⊘

Let's look up Charles.
Bucket 2 has wrong key.

Lookup: start at "correct" bucket, keep looking until we find the right key (or find an empty spot → key must be absent)

Linear probing hash table

If your bucket is taken, try the next one! (then the next, …)
→ Number of elements limited by the number of buckets.

(bucket)	name	phone
(0)	`"Allison"`	555–1212
(1)	⊘	⊘
(2)	`"Carol"`	555–1214
(3)	`"Charles"`	555–1217
(4)	⊘	⊘
⋮		
(24)	`"Youssef"`	555–1215
(25)	⊘	⊘

Let's look up Charles.
Bucket 2 has wrong key.
Bucket 3 has correct key!
We have Charles's number!

Lookup: start at "correct" bucket, keep looking until we find the right key (or find an empty spot → key must be absent)

Linear probing hash table

If your bucket is taken, try the next one! (then the next, …)
→ Number of elements limited by the number of buckets.

(bucket)	name	phone
(0)	`"Allison"`	555–1212
(1)	⊘	⊘
(2)	`"Carol"`	555–1214
(3)	`"Charles"`	555–1217
(4)	⊘	⊘
⋮		
(24)	`"Youssef"`	555–1215
(25)	⊘	⊘

Let's look up Carlos.

Lookup: start at "correct" bucket, keep looking until we find the right key (or find an empty spot → key must be absent)

Linear probing hash table

If your bucket is taken, try the next one! (then the next, …)
→ Number of elements limited by the number of buckets.

(bucket)	name	phone
(0)	`"Allison"`	555–1212
(1)	⊘	⊘
(2)	`"Carol"`	555–1214
(3)	`"Charles"`	555–1217
(4)	⊘	⊘
⋮		
(24)	`"Youssef"`	555–1215
(25)	⊘	⊘

Let's look up Carlos.
Bucket 2 has wrong key.

Lookup: start at "correct" bucket, keep looking until we find the right key (or find an empty spot → key must be absent)

Linear probing hash table

If your bucket is taken, try the next one! (then the next, …)
→ Number of elements limited by the number of buckets.

(bucket)	name	phone
(0)	`"Allison"`	555–1212
(1)	⊘	⊘
(2)	`"Carol"`	555–1214
(3)	`"Charles"`	555–1217
(4)	⊘	⊘
⋮		
(24)	`"Youssef"`	555–1215
(25)	⊘	⊘

Let's look up Carlos.
Bucket 2 has wrong key.
Bucket 3 has wrong key.

Lookup: start at "correct" bucket, keep looking until we find the right key (or find an empty spot → key must be absent)

Linear probing hash table

If your bucket is taken, try the next one! (then the next, …)
→ Number of elements limited by the number of buckets.

(bucket)	name	phone
(0)	`"Allison"`	555–1212
(1)	⊘	⊘
(2)	`"Carol"`	555–1214
(3)	`"Charles"`	555–1217
(4)	⊘	⊘
⋮		
(24)	`"Youssef"`	555–1215
(25)	⊘	⊘

Let's look up Carlos.
Bucket 2 has wrong key.
Bucket 3 has wrong key.
Bucket 4 is empty!
No Carlos in the dictionary!
Worst case: try all buckets!

Lookup: start at "correct" bucket, keep looking until we find the right key (or find an empty spot → key must be absent)

Linear probing hash table

If your bucket is taken, try the next one! (then the next, …)
→ Number of elements limited by the number of buckets.

(bucket)	name	phone
(0)	`"Allison"`	555–1212
(1)	⊘	⊘
(2)	`"Carol"`	555–1214
(3)	`"Charles"`	555–1217
(4)	⊘	⊘
⋮		
(24)	`"Youssef"`	555–1215
(25)	⊘	⊘

Let's insert Carlos.

Lookup: start at "correct" bucket, keep looking until we find the right key (or find an empty spot → key must be absent)

Linear probing hash table

If your bucket is taken, try the next one! (then the next, …)
→ Number of elements limited by the number of buckets.

(bucket)	name	phone
(0)	`"Allison"`	555–1212
(1)	⊘	⊘
(2)	`"Carol"`	555–1214
(3)	`"Charles"`	555–1217
(4)	⊘	⊘
⋮		
(24)	`"Youssef"`	555–1215
(25)	⊘	⊘

Let's insert Carlos.
Bucket 2 is taken.

Lookup: start at "correct" bucket, keep looking until we find the right key (or find an empty spot → key must be absent)

Linear probing hash table

If your bucket is taken, try the next one! (then the next, …)
→ Number of elements limited by the number of buckets.

(bucket)	name	phone
(0)	`"Allison"`	555–1212
(1)	⊘	⊘
(2)	`"Carol"`	555–1214
(3)	`"Charles"`	555–1217
(4)	⊘	⊘
⋮		
(24)	`"Youssef"`	555–1215
(25)	⊘	⊘

Let's insert Carlos.
Bucket 2 is taken.
Bucket 3 is taken.

Lookup: start at "correct" bucket, keep looking until we find the right key (or find an empty spot → key must be absent)

Linear probing hash table

If your bucket is taken, try the next one! (then the next, …)
→ Number of elements limited by the number of buckets.

(bucket)	name	phone
(0)	`"Allison"`	555–1212
(1)	⊘	⊘
(2)	`"Carol"`	555–1214
(3)	`"Charles"`	555–1217
(4)	`"Carlos"`	555–1219
⋮		
(24)	`"Youssef"`	555–1215
(25)	⊘	⊘

Let's insert Carlos.
Bucket 2 is taken.
Bucket 3 is taken.
Bucket 4 is empty!
Let's put Carlos there!

Lookup: start at "correct" bucket, keep looking until we find the right key (or find an empty spot → key must be absent)

Linear probing hash table

If your bucket is taken, try the next one! (then the next, …)
→ Number of elements limited by the number of buckets.

(bucket)	name	phone
(0)	`"Allison"`	555–1212
(1)	⊘	⊘
(2)	`"Carol"`	555–1214
(3)	`"Charles"`	555–1217
(4)	`"Carlos"`	555–1219
⋮		
(24)	`"Youssef"`	555–1215
(25)	⊘	⊘

Let's insert Carlos.
Bucket 2 is taken.
Bucket 3 is taken.
Bucket 4 is empty!
Let's put Carlos there!

Lookup: start at "correct" bucket, keep looking until we find the right key (or find an empty spot → key must be absent)

Fun fact: deletion becomes more complicated...
(See Chapter 8 of the textbook)

Stop! Question time!

Before we discuss performance...

Anything unclear so far?
Anything I missed?

Complexity?

Ok, now we've seen how hash tables work.

But *how well* do they work?

Last time: our benchmarks showed $\mathcal{O}(1)$ time(!)

How can we get that?

Complexity?

Ok, now we've seen how hash tables work.

But *how well* do they work?

Last time: our benchmarks showed $\mathcal{O}(1)$ time(!)

How can we get that?

Answer: it's subtle, and comes with caveats.

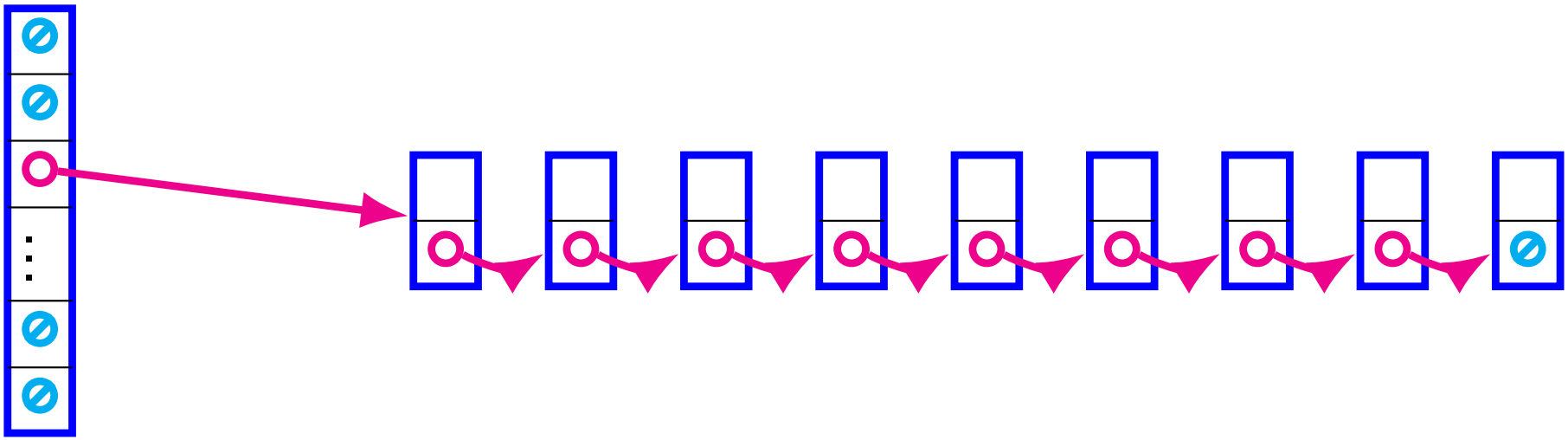
Let's consider the worst, best, and average case scenarios.

Worst-case scenario: Linear probing

(bucket)	name	phone
(0)	`"Allison"`	555–1212
(1)	`"Amanda"`	555–1213
(2)	`"Arnold"`	555–1214
(3)	`"Amadou"`	555–1215
(4)	`"Ahmed"`	555-1216
⋮		

- All n key-value pairs map to the *same* bucket.
- Need to probe $\mathcal{O}(n)$ buckets to find them.
- All operations are $\mathcal{O}(n)$. Yuck!

Worst-case scenario: Separate chaining



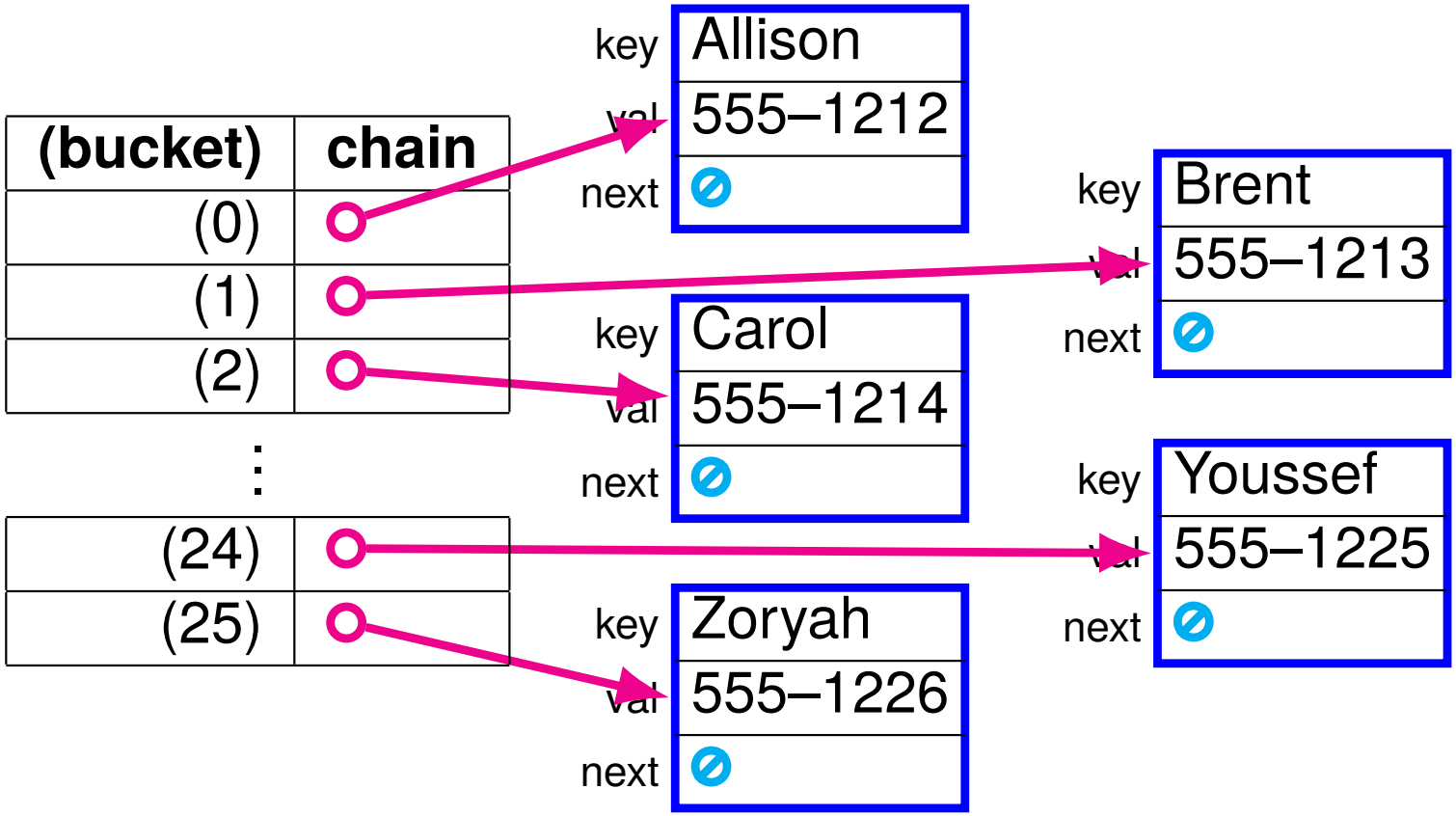
- All n key-value pairs map to the *same* bucket.
- Linked list of length n.
- All operations are $\mathcal{O}(n)$. Yuck!

Best-case scenario: Linear probing

(bucket)	name	phone
(0)	`"Allison"`	555–1212
(1)	`"Brent"`	555–1213
(2)	`"Carol"`	555–1214
(3)	`"Danielle"`	555–1215
(4)	`"Ernest"`	555-1216
⋮		
(24)	`"Youssef"`	555–1225
(25)	`"Zoryah"`	555–1226

- All n key-value pairs map to *different* buckets.
- Need to probe $\mathcal{O}(1)$ buckets to find them.
- All operations are $\mathcal{O}(1)$. Sweet!

Best-case scenario: Separate chaining



- All n key-value pairs map to *different* buckets.
- **Also**, enough buckets to have one key-value pair per.
- Linked lists of length 1.
- All operations are $\mathcal{O}(1)$. Sweet!

Average-case scenario?

Either extreme is an unlikely scenario.

What makes the good cases good?

1. No (or few) hash collisions
2. Enough buckets for the number of keys.

Average-case scenario?

Either extreme is an unlikely scenario.

What makes the good cases good?

1. No (or few) hash collisions
2. Enough buckets for the number of keys.

How can we design our hash tables to make bad cases less likely and good cases more likely?

1. Choose a hash function that makes collision less likely.
2. Make sure we always have enough buckets.

If we play our cards right, we can get the *average* case to be on par with the best case!

- Worst case still exists, but unlikely
- We won't lose sleep over it

The Quest for a Good Hash Function

Our h_1 hash function is absolute garbage

h_1 is horrendously prone to collisions

- Using the first letter limits us to 26 buckets
  - With 27+ entries, collisions are **guaranteed**!
- Some letters have more names than others
  - Buckets are likely to be uneven
  - So most keys will map to buckets with lots of other keys!
- Etc. (It's **really** bad.)

Let's find a better one.

Hash function criteria

The only *necessary* conditions for a hash function:

1. Must output natural numbers
   - Otherwise we can't index arrays with its results!

2. A given key must always map to the same hash code (*deterministic*)
   - Otherwise, we won't be able to retrieve things we store!

So our "first letter" hash function h_1 is valid, albeit really bad

Hash function criteria

So what makes a **good** hash function?

- **Large output range**: to allow a large number of buckets
  - Say, $0 \ldots 2^{64} - 1$ rather than $0 \ldots 25$
  - Use modulo to "scale" it down to # of buckets in the table.
    E.g., 100 buckets, $h($`"Charles"`$)$ = 172350
    $\rightarrow$ 172350 modulo 100 $\rightarrow$ bucket 50

Hash function criteria

So what makes a **good** hash function?

- **Large output range**: to allow a large number of buckets
  - Say, $0 \ldots 2^{64} - 1$ rather than $0 \ldots 25$
  - Use modulo to "scale" it down to # of buckets in the table.
    E.g., 100 buckets, $h($`"Charles"`$)$ = 172350
    $\rightarrow$ 172350 modulo 100 $\rightarrow$ bucket 50
- **Uniform**: hashes are spread uniformly over their range
  - To spread key-value pairs out across all buckets
  - $\rightarrow$ makes collisions less likely

Hash function criteria

So what makes a **good** hash function?

- **Large output range**: to allow a large number of buckets
  - Say, $0 \ldots 2^{64} - 1$ rather than $0 \ldots 25$
  - Use modulo to "scale" it down to # of buckets in the table.
    E.g., 100 buckets, $h($`"Charles"`$)$ = 172350
    $\rightarrow$ 172350 modulo 100 $\rightarrow$ bucket 50
- **Uniform**: hashes are spread uniformly over their range
  - To spread key-value pairs out across all buckets
  - $\rightarrow$ makes collisions less likely
- **Diffusion**: similar inputs produce very different hashes
  - E.g. $h($`"George"`$) = 34$ but $h($`"Georges"`$) = 165293787$
  - Ideally, 1 bit diff. in input $\rightarrow$ half the bits diff. in output
  - Real inputs are likely to share similarities
  - E.g., a lot of "Smith"s in the phone book

Hash function criteria

So what makes a **good** hash function?

- **Large output range**: to allow a large number of buckets
  - Say, $0 \ldots 2^{64} - 1$ rather than $0 \ldots 25$
  - Use modulo to "scale" it down to # of buckets in the table.
    E.g., 100 buckets, $h($`"Charles"`$)$ = 172350
    $\rightarrow$ 172350 modulo 100 $\rightarrow$ bucket 50
- **Uniform**: hashes are spread uniformly over their range
  - To spread key-value pairs out across all buckets
  - $\rightarrow$ makes collisions less likely
- **Diffusion**: similar inputs produce very different hashes
  - E.g. $h($`"George"`$) = 34$ but $h($`"Georges"`$) = 165293787$
  - Ideally, 1 bit diff. in input $\rightarrow$ half the bits diff. in output
  - Real inputs are likely to share similarities
  - E.g., a lot of "Smith"s in the phone book
- Etc. (This is a very deep topic.)
  - **Bonus**: good hash functions useful beyond hash tables!
    (cryptography, checksums, bloom filters (later))

Good news!

DSSL2 comes with a way to generate excellent hash functions.

```
#lang dssl2

import sbox_hash

let my_hash_function = make_sbox_hash()
my_hash_function("George")  # => 12954263217209680626
my_hash_function("Georges") # => 17471623022190201144

# can create multiple different hash functions
# e.g., for multiple hash tables
# each call to make_sbox_hash generates a new, randomly
# generated (but deterministic!) hash function
let my_hash_function2 = make_sbox_hash()
my_hash_function2("George")  # => 9101004326490879 33
my_hash_function2("Georges") # => 16264795133863970133
```

Maintaining Enough Buckets

Load

For good performance, we can't let the table get too full

- **Separate chaining**: the length of the chains is $\mathcal{O}(n)$
- **Linear probing**: the length of the search is $\mathcal{O}(n)$

Load

For good performance, we can't let the table get too full

- **Separate chaining**: the length of the chains is $\mathcal{O}(n)$
- **Linear probing**: the length of the search is $\mathcal{O}(n)$

We measure how full a hash table is using its *load factor*:

$$\mathrm{L} = \frac{n}{k}$$

- n: number of dictionary elements (key-value pairs)
- k: number of buckets

Lower is better (up to a point).

Load

For good performance, we can't let the table get too full

- **Separate chaining**: the length of the chains is $\mathcal{O}(n)$
- **Linear probing**: the length of the search is $\mathcal{O}(n)$

We measure how full a hash table is using its *load factor*:

$$\mathrm{L} = \frac{n}{k}$$

- n: number of dictionary elements (key-value pairs)
- k: number of buckets

Lower is better (up to a point).

Good upper bounds in practice, derived empirically:

- For separate chaining, want $\mathrm{L} < 2$
- For linear probing, want $\mathrm{L} < 0.75$

General idea: *expected* **constant** # elements / bucket.

Resizing

As we insert more key-value pairs, load factor increases.
When the load factor gets too high, we need to *grow* the table
I.e., create a bigger vector; like we did for dynamic arrays.

- Requires rehashing everything!
  - With new # of buckets, elements may not map to the same bucket as before!
  - And we still want to be able to find them
  - Even more work than copying all the elements of an array!

Example:

- Start with 100 buckets. $h($`"George"`$) = 2134$
  - George goes to bucket: **???**

Resizing

As we insert more key-value pairs, load factor increases.
When the load factor gets too high, we need to *grow* the table
I.e., create a bigger vector; like we did for dynamic arrays.

- Requires rehashing everything!
  - With new # of buckets, elements may not map to the same bucket as before!
  - And we still want to be able to find them
  - Even more work than copying all the elements of an array!

Example:

- Start with 100 buckets. $h($`"George"`$) = 2134$
  - George goes to bucket: 2134 modulo 100 = 34

Resizing

As we insert more key-value pairs, load factor increases.
When the load factor gets too high, we need to *grow* the table
I.e., create a bigger vector; like we did for dynamic arrays.

- Requires rehashing everything!
  - With new # of buckets, elements may not map to the same bucket as before!
  - And we still want to be able to find them
  - Even more work than copying all the elements of an array!

Example:

- Start with 100 buckets. $h($`"George"`$) = 2134$
  - George goes to bucket: 2134 modulo 100 = 34
- Grow to 200 buckets
  - Where do we look for George now? **???**

Resizing

As we insert more key-value pairs, load factor increases.
When the load factor gets too high, we need to *grow* the table
I.e., create a bigger vector; like we did for dynamic arrays.

- Requires rehashing everything!
  - With new # of buckets, elements may not map to the same bucket as before!
  - And we still want to be able to find them
  - Even more work than copying all the elements of an array!

Example:

- Start with 100 buckets. $h($`"George"`$) = 2134$
  - George goes to bucket: 2134 modulo 100 = 34
- Grow to 200 buckets
  - Where do we look for George now? 2134 modulo 200 = 134

Resizing

As we insert more key-value pairs, load factor increases.
When the load factor gets too high, we need to *grow* the table
I.e., create a bigger vector; like we did for dynamic arrays.

- Requires rehashing everything!
  - With new # of buckets, elements may not map to the same bucket as before!
  - And we still want to be able to find them
  - Even more work than copying all the elements of an array!

Example:

- Start with 100 buckets. $h($`"George"`$) = 2134$
  - George goes to bucket: 2134 modulo 100 = 34
- Grow to 200 buckets
  - Where do we look for George now? 2134 modulo 200 = 134
  - Look for George in bucket 134. Oops, not there!
- Need to hash the key `"George"` again
  - Then put George in bucket 134 of the new table

Resizing

Doubling the size strikes a good balance

- Not wasting too much space on unused buckets
- Not constantly having to grow and rehash
- Turns out doubling is also what we want for dynamic arrays!
  - We'll see the math for that one in a few weeks

Rehashing is a $\mathcal{O}(n)$ operation, in all cases

- Worst, best, average, doesn't matter
- Need to re-insert every key-value pair
- Another caveat to the $\mathcal{O}(1)$

Complexity of lookup

More good news!

- If you have enough buckets
  (i.e., load factor is constant).
- And your hash function is good
  (i.e, elements are spread uniformily across buckets).

Then you'll have $\mathcal{O}(1)$ elements per bucket *on average*.

Complexity of lookup

More good news!

- If you have enough buckets
  (i.e., load factor is constant).
- And your hash function is good
  (i.e, elements are spread uniformily across buckets).

Then you'll have $\mathcal{O}(1)$ elements per bucket *on average*.

Thus, lookup will have an *average case* asymptotic complexity of $\mathcal{O}(1)$! Pretty great.

- I.e., when averaging over all possible inputs, $\mathcal{O}(1)$

Complexity of lookup

More good news!

- If you have enough buckets
  (i.e., load factor is constant).
- And your hash function is good
  (i.e, elements are spread uniformily across buckets).

Then you'll have $\mathcal{O}(1)$ elements per bucket *on average*.

Thus, lookup will have an *average case* asymptotic complexity of $\mathcal{O}(1)$! Pretty great.

- I.e., when averaging over all possible inputs, $\mathcal{O}(1)$
- **But**, worst case is still $\mathcal{O}(n)$. It's very unlikely, though, so we won't worry too much about it.
- The actual analysis is out of scope for us.
  - (If you're curious, CLRS pages 227 and 241.)

Complexity of lookup

More good news!

- If you have enough buckets
  (i.e., load factor is constant).
- And your hash function is good
  (i.e, elements are spread uniformily across buckets).

Then you'll have $\mathcal{O}(1)$ elements per bucket *on average*.

Thus, lookup will have an *average case* asymptotic complexity of $\mathcal{O}(1)$! Pretty great.

- I.e., when averaging over all possible inputs, $\mathcal{O}(1)$
- **But**, worst case is still $\mathcal{O}(n)$. It's very unlikely, though, so we won't worry too much about it.
- The actual analysis is out of scope for us.
  - (If you're curious, CLRS pages 227 and 241.)

Non-resizing insertion is also $\mathcal{O}(1)$ on average, same reason.

But resizing insertion is $\mathcal{O}(n)$ always. (But see lecture 17.)

More on hash tables

Visualizations

- Separate chaining: `https://www.cs.usfca.edu/~galles/visualization/OpenHash.html`
- Open addressing: `https://www.cs.usfca.edu/~galles/visualization/ClosedHash.html`

He calls them different names, but they're the same thing.

History

- Nice overview: `https://spectrum.ieee.org/hans-peter-luhn-and-the-birth-of-the-hashing-algorithm`

Heads up: Required reading

Next time, we will begin our discussion of graphs.

- We will not cover graph basics in lecture
- Instead, you will need to read Chapter 10 of the textbook
  - Do this before lecture, or you will be completely lost!
  - Also, one of the worksheets is about that (due tonight!)
- We will start next lecture with some time for Q&A about it

If we have time: linear probing hash table in DSSL2

Next time: Graphs